



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE SOFTWARE

TEMA:

Practica Experimental 1

GRUPO:

CAJAS IBARRA IRVIN MARCELO
PANAMA MURILLO MOISES ANTONIO
TAIPE MORA ZAIDA MELISSA

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web – A – [5to Nivel]

Quevedo – Ecuador 2026

UNIDAD 1: Fundamentos y Diseño Web

Tema 1: Fundamentos de las Aplicaciones Web

1.1 Definición y clasificación de aplicaciones web

Una aplicación web es software que corre en un servidor y el usuario accede desde el navegador sin instalar nada. La diferencia con una app nativa no es solo técnica: implica decisiones distintas de arquitectura, seguridad y mantenimiento.

La clasificación más útil parte de lo que el usuario hace. Las aplicaciones *estáticas* sirven contenido fijo sin procesamiento en el servidor; las *dinámicas* generan respuestas personalizadas en cada solicitud. Las SPA (*Single-Page Applications*) complican ese esquema porque su procesamiento del lado del cliente puede ser tan intenso como el de cualquier programa de escritorio, aunque sigan dependiendo de un servidor para datos y sesiones [1]. Las PWA (*Progressive Web Apps*) añaden capacidades offline mediante Service Workers y pueden instalarse en el dispositivo; a diferencia de las SPA, mejoran la indexabilidad y la resiliencia ante fallos de red, aunque introducen complejidad en la gestión de caché [1]. Las aplicaciones de uso interno empresarial tienen requisitos de seguridad distintos a los de una app pública [2], y los sistemas basados en microservicios, si bien mejoran la escalabilidad, incrementan la superficie de ataque y la complejidad operativa [3].

1.2 Arquitectura cliente-servidor y ciclo de vida de una solicitud HTTP/HTTPS

El navegador masificó el modelo cliente-servidor: cualquier máquina con acceso a internet puede actuar como terminal sin preparación previa. El esquema clásico de tres capas — presentación, lógica y datos— permite escalar cada parte de forma independiente, aunque con la nube y los microservicios el “servidor” dejó de ser una máquina concreta para convertirse en un conjunto distribuido de servicios [3]. Escala mejor, pero la complejidad no desapareció; solo cambió de lugar.

El ciclo completo de una solicitud web comienza con una consulta DNS para traducir el dominio a una dirección IP. Si la conexión es HTTPS, se ejecuta el *TLS handshake*: el servidor presenta su certificado, el cliente lo verifica con una autoridad certificadora y ambas partes negocian las claves de cifrado. Solo entonces comienza el intercambio HTTP. Los métodos definen la semántica de cada operación: **GET** recupera un recurso sin modificar el estado del servidor; **POST** crea uno nuevo; **PUT** reemplaza un recurso completo; **PATCH** lo

modifica parcialmente; y **DELETE** lo elimina. Las peticiones **GET** son cacheables y se guardan en el historial del navegador, mientras que **POST** y **PUT** no lo son por defecto. Los códigos de estado estructuran la respuesta: 2xx indican éxito, 3xx redirecciones, 4xx errores del cliente y 5xx errores del servidor [2].

Las cabeceras de seguridad instruyen al navegador sobre cómo manejar el contenido: **Strict-Transport-Security** (HSTS) fuerza el uso de HTTPS en solicitudes futuras; **Content-Security-Policy** (CSP) restringe los recursos externos que la página puede cargar, mitigando ataques XSS; y **X-Frame-Options** impide incrustar la página en un `<iframe>` ajeno, previniendo *clickjacking*. Ignorar estas cabeceras es una de las causas más comunes de vulnerabilidades en aplicaciones web [2].

1.3 Evolución histórica de la web

La propuesta de Berners-Lee en 1989 era modesta: documentos enlazados para compartir información entre investigadores. La primera etapa fue básicamente de solo lectura. Lo que cambió el modelo fue la maduración de JavaScript y la posibilidad de hacer peticiones al servidor sin recargar la página, habilitando interfaces en tiempo real. Sobre esa base surgieron los blogs, las redes sociales y todo lo que se conoce como Web 2.0 [1]. Lo que viene después sigue siendo debate abierto: hay quienes apuntan a una web donde las máquinas entienden el contenido semánticamente, y quienes la asocian con sistemas donde los datos no dependan de unas pocas empresas [3].

1.4 Ventajas y desventajas de las aplicaciones web

La ventaja más clara es que un solo despliegue llega a cualquier sistema operativo sin tocar el código, y las actualizaciones alcanzan a todos los usuarios al mismo tiempo. El lado malo es menos discutido: sin conexión la aplicación no funciona, las tareas con mucho procesamiento local siguen siendo más lentas que en una app nativa, y cualquier persona con un navegador puede intentar interactuar de formas no previstas, lo que convierte la seguridad en un tema permanente [2]. A eso se suma una dependencia difícil de controlar: si el proveedor de nube falla o cambia sus condiciones, el equipo de desarrollo no tiene alternativa inmediata.

1.5 Diseño web inclusivo, accesibilidad y marco legal en Ecuador

Los errores de accesibilidad más comunes en sitios web son técnicamente simples: imágenes sin descripción de texto, formularios sin etiquetas, colores con contraste insuficiente. El pro-

blema es que aparecen una y otra vez en revisiones de sitios educativos y universitarios de todo el mundo [4], [5]. No es falta de conocimiento; es que la accesibilidad no entra en el flujo de trabajo normal.

Las WCAG 2.1 (*Web Content Accessibility Guidelines*) organizan los criterios de accesibilidad en cuatro principios conocidos como POUR. El principio de **Perceptibilidad** establece que toda la información debe poder presentarse de formas que el usuario pueda percibir: imágenes deben tener texto alternativo (criterio 1.1.1, nivel A), los videos deben incluir subtítulos (1.2.2, A) y el contraste mínimo entre texto y fondo debe ser de 4,5:1 para texto normal (1.4.3, nivel AA). El principio de **Operabilidad** requiere que toda la funcionalidad sea accesible desde el teclado (2.1.1, A) y que los usuarios dispongan de tiempo suficiente para interactuar con el contenido (2.2.1, A). El principio de **Comprensibilidad** exige que el idioma de la página esté declarado (3.1.1, A), que las etiquetas de los formularios sean descriptivas (3.3.2, A) y que los errores se identifiquen y describan con claridad (3.3.1, A). El principio de **Robustez** demanda que el contenido pueda ser interpretado de forma fiable por tecnologías asistivas actuales y futuras, lo que implica usar HTML semántico válido y atributos ARIA correctamente (4.1.2, A) **W3CWCAG21**.

En el contexto ecuatoriano, la accesibilidad web no es solo una buena práctica: tiene respaldo legal. La *Ley Orgánica de Discapacidades* (LOD) establece en su artículo 19 el derecho de las personas con discapacidad a acceder a la información y comunicaciones en igualdad de condiciones. La *Norma Técnica Ecuatoriana INEN 2620* define los requisitos de accesibilidad para los servicios y productos de tecnología de la información, incluyendo sitios web de instituciones públicas. El incumplimiento de estas normas en aplicaciones de uso público puede derivar en observaciones de organismos de control. Para VetSystem, la implementación de WCAG 2.1 nivel AA no es solo un requisito académico; es la base técnica que garantiza el cumplimiento de este marco normativo [5].

Diseñar de forma inclusiva no es cumplir un checklist. Implica tomar decisiones que funcionen para el mayor rango de usuarios desde el inicio: personas con problemas de visión o movilidad, pero también quienes acceden desde conexiones lentas o dispositivos económicos. Cuando se diseña pensando en esos casos límite, el resultado suele ser más claro para todos [6].

Las herramientas automáticas de auditoría de accesibilidad tienen capacidades y limitaciones distintas, y ninguna es suficiente por sí sola. **Lighthouse** (integrado en Chrome DevTools) realiza un análisis estático de la página cargada y puntúa la accesibilidad sobre 100; detecta problemas de contraste, ausencia de etiquetas en formularios, falta de atributos **alt** y orden incorrecto de encabezados, pero no puede evaluar flujos de interacción ni contenido dinámico

generado después de la carga [6]. **WAVE** (*Web Accessibility Evaluation Tool*) presenta los errores de forma visual sobre la página, lo que facilita localizar exactamente qué elemento tiene el problema; es especialmente útil para identificar errores de contraste, etiquetas faltantes y estructura semántica deficiente, aunque tampoco alcanza el contenido generado dinámicamente por JavaScript tras la interacción del usuario [4]. **AXE DevTools** se integra directamente en el panel de desarrollador del navegador y reporta únicamente los problemas con alta certeza de ser un error real, minimizando los falsos positivos; su ventaja sobre las otras dos herramientas es que puede ejecutarse sobre estados específicos de la aplicación después de interacciones del usuario, lo que lo hace más adecuado para auditar SPAs [6]. La combinación de las tres herramientas es necesaria porque sus conjuntos de reglas no son idénticos: un problema detectado por WAVE puede no aparecer en Lighthouse, y viceversa. Además, ninguna herramienta automática puede reemplazar las pruebas manuales con lectores de pantalla reales (NVDA, JAWS).

Tema 2: HTML y HTML5 para el Desarrollo Web

2.1 Fundamentos de HTML

HTML no ejecuta código; describe qué es cada parte de un documento. Esa separación entre estructura, estilo y comportamiento define cómo trabajan juntos HTML, CSS y JavaScript. La historia del lenguaje incluye un período caótico en los años 90, cuando Internet Explorer y Netscape agregaban etiquetas propias que el otro no reconocía [1]. Lo que quedó es un parser que acepta código mal escrito e intenta adivinar la intención del autor, porque demasiados sitios reales funcionaban con errores que ningún navegador podía rechazar.

2.2 Elementos semánticos de HTML5

Antes de HTML5 todo se estructuraba con `<div>` y un atributo `id`. HTML5 introdujo etiquetas que nombran lo que contienen: `<nav>`, `<header>`, `<main>`, `<article>`, `<aside>` y `<footer>`. El efecto más directo fue en accesibilidad: un lector de pantalla puede ofrecer atajos para saltar al contenido principal sin trabajo extra del desarrollador [4]. Según la especificación WHATWG, `<article>` representa contenido autocontenido que tendría sentido fuera del documento, mientras que `<section>` agrupa contenido temático relacionado pero no independiente. `<main>` debe ser único por página y `<aside>` alberga contenido prescindible. La jerarquía `<h1>`–`<h6>` no debe romperse: saltar niveles confunde a los lectores de pantalla que la usan para navegar **W3CWCAG21**.

WAI-ARIA complementa la semántica cuando los elementos nativos no son suficientes, pero el principio rector es “no ARIA es mejor que mal ARIA”: un `role` incorrecto comunica información falsa a las tecnologías asistivas. Los roles más relevantes son `role="dialog"` para modales, `role="alert"` para mensajes urgentes y `role="status"` para actualizaciones no urgentes. Los estados `aria-expanded` y `aria-checked` comunican el estado de componentes interactivos, mientras que `aria-label`, `aria-labelledby` y `aria-describedby` nombran elementos sin etiqueta visible o los asocian con su mensaje de error **W3CARIA12**.

2.3 Formularios HTML5 y validación nativa

HTML5 sumó tipos de entrada específicos —`email`, `tel`, `date`, `number`, `range`— y atributos como `required` o `pattern` que permiten validación básica sin JavaScript. En móviles, el tipo de `<input>` determina el teclado mostrado: `type="tel"` activa el numérico, `type="email"` incluye el símbolo @, lo que impacta directamente en la tasa de completado. La validación nativa no requiere JavaScript y se integra con lectores de pantalla mediante `aria-invalid`, pero sus mensajes de error varían entre navegadores y no cubre lógica de negocio compleja. Más importante: nunca puede sustituir la validación en el servidor, porque cualquier dato del navegador puede haber sido manipulado [2]. El patrón correcto combina las tres capas: HTML5 para retroalimentación inmediata, JavaScript para lógica avanzada y servidor como defensa definitiva. Existe trabajo reciente en usar inteligencia artificial para detectar errores mientras el usuario escribe [7], aunque su integración en producción plantea preguntas sobre la privacidad de los datos de comportamiento [8].

2.4 Multimedia y APIs de HTML5

Flash dominó la web porque era la única opción para video, audio y animaciones. HTML5 lo desplazó al introducir `<audio>` y `<video>` como elementos nativos, con mejor rendimiento y sin sus problemas de seguridad. La parte más relevante para el desarrollo moderno fue el conjunto de APIs habilitadas: geolocalización, Web Workers y el elemento `<canvas>`. Para la persistencia de datos, `localStorage` almacena hasta 5 MB y persiste entre sesiones; `sessionStorage` tiene la misma capacidad pero se limpia al cerrar la pestaña; las cookies se limitan a 4 KB y se envían en cada petición HTTP. Tanto `localStorage` como `sessionStorage` son vulnerables a XSS; las cookies pueden protegerse con `HttpOnly` para hacerlas inaccesibles desde JavaScript [2]. El `<canvas>` trabaja con píxeles y no con elementos del DOM, por lo que carece de estructura semántica para lectores de pantalla y sus errores visuales son los más difíciles de detectar con herramientas automáticas **Macklon2023**.

2.5 XML y familias de lenguajes de marcado

XML permite que dos sistemas intercambien datos estructurados sin un formato propietario, pero cualquier error de sintaxis invalida todo el archivo, a diferencia del HTML tolerante. El experimento de XHTML intentó aplicar ese rigor a HTML; en la práctica, que un solo tag mal cerrado rompiera la página fue demasiado para el ecosistema existente y no prosperó [1]. XML se quedó donde sí funciona: intercambio de datos, configuración y dialectos como SVG, cuyos elementos forman parte del DOM, se estilizan con CSS y son accesibles para lectores de pantalla [4]. La decisión entre SVG y `<canvas>` no es de gusto: SVG es correcto cuando el gráfico necesita ser accesible o manipulable; `<canvas>` es preferible para renderizar miles de objetos en tiempo real **Macklon2023**.

Tema 3: CSS para el Diseño y la Presentación Web

3.1 Fundamentos de CSS

Cascading Style Sheets (CSS) estableció la separación entre estructura HTML y presentación visual, pero su aparente sencillez esconde una complejidad real: el mecanismo de cascada y la especificidad de selectores generan conflictos difíciles de gestionar a escala [9]. La crítica más recurrente es que la cascada obliga al uso excesivo de `!important` para sobrescribir estilos heredados, deteriorando la mantenibilidad. Frente a esto, propuestas como las Constraint Cascading Style Sheets (CCSS) plantean un formalismo de restricciones matemáticas que permitiría relaciones declarativas complejas —como mantener dos columnas al mismo ancho mínimo— y una degradación más elegante cuando los recursos del sistema del usuario no coinciden con los del diseñador [9]. Un ejemplo básico ilustra la especificidad: `h1 { font-size: 13pt; }` frente a `#header { color: blue; }`, donde el identificador siempre gana [10].

3.2 Modelo de caja y maquetación CSS

El modelo de caja —contenido, *padding*, *border* y *margin*— es el concepto arquitectónico central de la maquetación web. Cualquier alteración en estas propiedades dispara procesos de *reflow* y *repaint* en el motor de renderizado, con un coste real en CPU y GPU: estudios demuestran que la maquetación DOM con más de 500 elementos posicionados provoca degradación severa de fotogramas [11]. La propiedad `box-sizing: border-box` de CSS3 resolvió el problema histórico de que el padding y el border se sumaban al ancho definido. Para interfaces multimedia complejas, modelos de optimización por niveles de interacción resultan más

adecuados que el modelo estándar [12], y la filosofía *Mise en Place* —componentes atómicos y reutilizables— reduce la duplicación de declaraciones que afecta a una parte significativa del código en producción [13], [14].

3.3 Diseño Flexbox y CSS Grid

Flexbox gestiona la distribución en una sola dimensión; CSS Grid lo hace en dos. Esta diferencia define cuándo usar cada uno: Flexbox es ideal para componentes como barras de navegación donde importa el eje, mientras que Grid es superior para el layout general de la página, con áreas nombradas y control explícito de filas y columnas [15]. Frameworks como Bulma y UIKit aprovechan Flexbox para alineación eficiente sin porcentajes manuales [16], [17]. La crítica técnica señala que Flexbox genera inconsistencias en rejillas cuando los elementos tienen tamaños de contenido distintos, mientras Grid separa mejor el orden del DOM de la posición visual, beneficiando la accesibilidad [13]. En la práctica, la combinación es la norma: Grid para el andamiaje exterior, Flexbox para los componentes internos [10].

```
/* Flexbox */
.container { display: flex; justify-content: center; align-items:
    center; }
/* Grid */
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10
    px; }
```

3.4 Diseño responsivo y mobile-first

El diseño responsivo se fundamenta en rejillas fluidas, imágenes flexibles y *media queries* [18]. La filosofía *mobile-first* va más allá: obliga a comenzar por la pantalla más restrictiva, priorizando contenido esencial y cargando solo los estilos mínimos iniciales [16]. Es imprescindible configurar el *viewport* con `<meta name="viewport" content="width=device-width, initial-scale=1">` para evitar que el navegador escale la versión de escritorio [10]. Frente al diseño adaptativo —*breakpoints* fijos y layouts predefinidos—, el responsivo ofrece mayor fluidez. Sin embargo, ambos deben considerar la accesibilidad: el diseño fluido puede afectar a usuarios con discapacidades si no se usan unidades relativas (`em`, `rem`, `%`) en lugar de píxeles absolutos [10].

3.5 CSS3 avanzado y preprocesadores

Los preprocesadores como SASS, LESS y Stylus incorporan variables, *mixins*, anidamiento y herencia al flujo de trabajo CSS [14]. SASS destaca por su popularidad y la filosofía DRY (*Don't Repeat Yourself*): definir un color en un solo lugar y propagarlo a cientos de elementos simplifica mantenimientos masivos. Los postprocesadores como PostCSS complementan este flujo automatizando la inserción de prefijos de vendor mediante Autoprefixer [14]. La diferencia clave es que SASS actúa en la fase de autoría mientras PostCSS optimiza post-desarrollo. Una tendencia emergente es CSS-in-JS, que vincula estilos directamente al estado del componente JavaScript eliminando problemas de alcance global, aunque con coste adicional en el cliente [13]. La elección depende de la madurez del equipo; SASS sigue siendo la opción más estable y documentada [14].

Tema 4: JavaScript para Aplicaciones Web

4.1 Fundamentos de JavaScript

JavaScript evolucionó de un lenguaje de *scripting* creado en diez días a la infraestructura de la web moderna, impulsado por motores como V8 que habilitaron su uso en el servidor vía Node.js [19], [20]. Su naturaleza de hilo único y el modelo de herencia por prototipos lo diferencian de Java o C++: esta flexibilidad es eficiente pero introduce trampas en la gestión del ámbito y la asincronía [20]. Para tareas intensivas como simulaciones, puede ser superado por WebAssembly, que opera más cerca del hardware [11]. Independientemente del framework utilizado, el dominio del *event loop*, los tipos de datos y los operadores sigue siendo la base de cualquier aplicación web; JavaScript está presente en más del 95 % de los sitios actuales [21], [14].

4.2 Estructuras de control y funciones

ES6 resolvió problemas históricos de `var` —alcance de función y *hoisting* impredecible— con `let` y `const`, que tienen alcance de bloque; `const` promueve la inmutabilidad al impedir reasignaciones [21]. Las estructuras de control tradicionales (`for`, `while`, `do...while`) conviven con métodos modernos como `forEach`, más expresivos para arreglos. Las *arrow functions* simplifican la sintaxis y no vinculan su propio `this`, crítico en callbacks y métodos de clase. El siguiente ejemplo ilustra la diferencia de alcance:

```
if (true) { var x = 5; let y = 10; }
```

```
console.log(x); // 5
console.log(y); // ReferenceError
```

En frameworks como React, las funciones son la unidad básica de construcción: los *Functional Components* con *Hooks* han desplazado a las clases por su menor complejidad y mejor testabilidad [21], [13].

4.3 Manipulación del DOM y eventos

El DOM es la interfaz mediante la cual JavaScript modifica contenido, estilos y estructura en tiempo real, pero cada modificación puede desencadenar *reflow* y *repaint*. Benchmarks muestran estabilidad con 100 objetos animados y degradación severa a partir de 500, especialmente con cambios de color y transparencia [11]. Los frameworks modernos mitigan esto con el *Virtual DOM*, que agrupa cambios antes de aplicarlos al DOM real [21]. La interactividad se gestiona mediante `addEventListener`, cuyo modelo asíncrono procesa eventos en cola:

```
const btn = document.querySelector('#miBtn');
btn.addEventListener('click', () => {
  document.body.style.backgroundColor = 'red';
});
```

La distinción clave es DOM vs. Canvas: el DOM mantiene semántica accesible para lectores de pantalla y buscadores; Canvas manipula píxeles directamente, siendo más eficiente para miles de objetos simultáneos pero perdiendo accesibilidad nativa [11].

4.4 JavaScript moderno (ES6+) y programación asíncrona

El estándar moderno transformó la asincronía: de *callbacks* anidados —el infame *callback hell*— a Promesas y finalmente a `async/await`, que permite escribir código asíncrono con apariencia secuencial [19], [20]. ES6+ también introdujo módulos nativos (`import/export`), desestructuración, literales de plantilla y clases [20], [21]. Un aspecto crítico en aplicaciones de simulación es la prevención de *race conditions* mediante sincronización de acceso a recursos compartidos [19]. Este ecosistema sustenta tanto SPAs como aplicaciones con renderizado en servidor comparables a las de escritorio [21].

Informe de Práctica: VetSystem

1. Descripción del Proyecto

VetSystem es una aplicación web de una sola página (SPA) desarrollada como práctica experimental de Desarrollo Web Frontend. El sistema permite gestionar el registro de mascotas en una clínica veterinaria, incluyendo datos del propietario, información clínica, programación de citas y seguimiento de vacunas.

La aplicación fue construida íntegramente con tecnologías web estándar: HTML5 semántico y accesible, CSS3 con variables y diseño responsivo, y JavaScript ES6+ con arquitectura modular. Los datos se persisten en el navegador mediante Web Storage y se integra una API externa para enriquecer la experiencia.

2. Paso 1 — Configuración del Entorno

2.1 Repositorio GitHub

Se creó un repositorio compartido en GitHub para el trabajo colaborativo del equipo. Cada integrante realizó commits individuales desde su rama, garantizando trazabilidad del trabajo.

URL del repositorio: <https://github.com/Zaida-tm18/Practicas-Experimentales.git>

2.2 Extensiones de VS Code

Se instalaron las siguientes extensiones en Visual Studio Code para garantizar un flujo de trabajo profesional:

- **Live Server** (Ritwick Dey) — recarga automática del navegador en localhost:5500
- **Prettier - Code formatter** — formateo consistente del código
- **ESLint** (Microsoft) — análisis estático de JavaScript
- **axe Accessibility Linter** (Deque Systems) — validación WCAG en el editor
- **GitLens** (GitKraken) — historial de commits en el editor

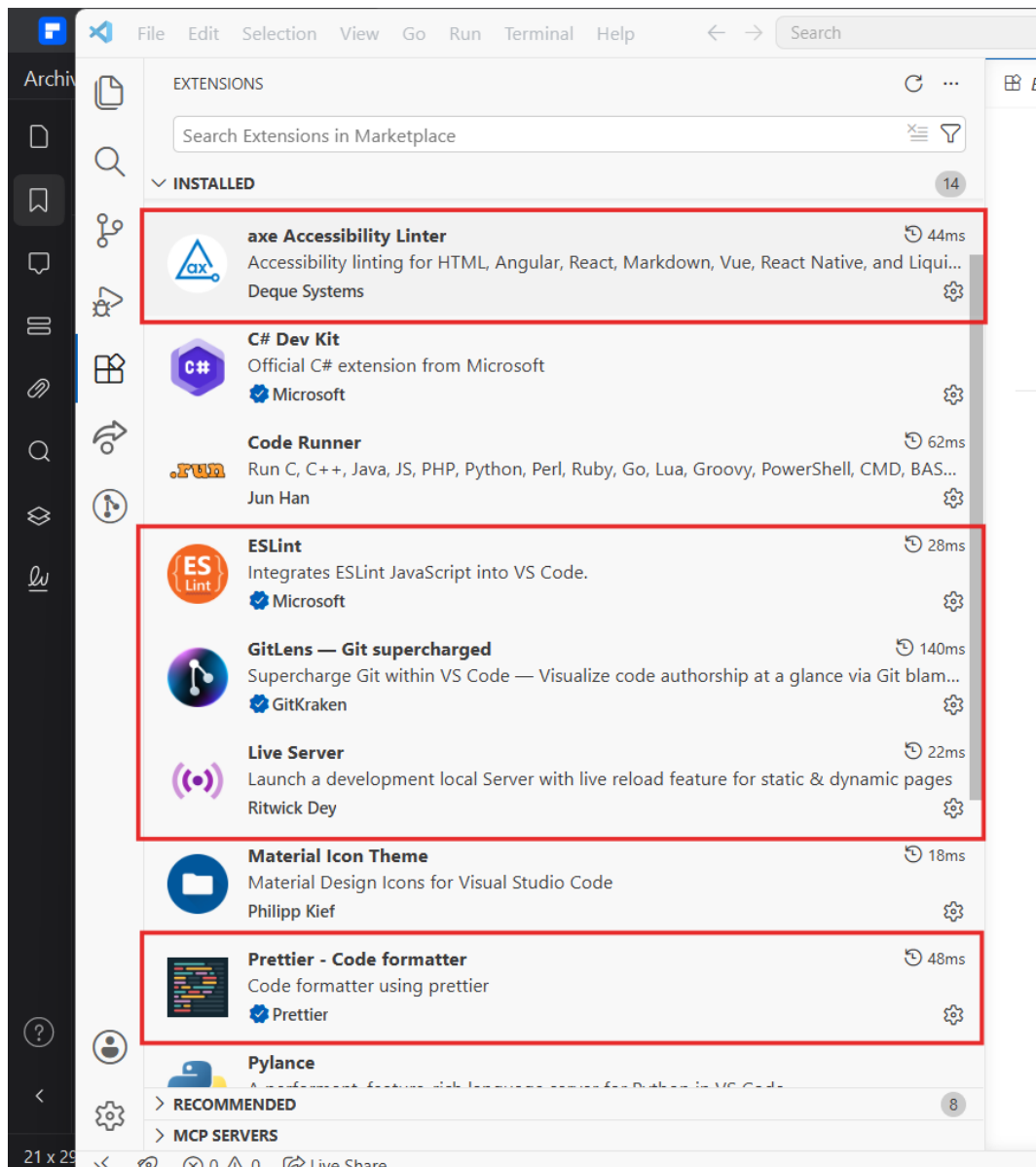


Figura 1: Extensiones instaladas en VS Code

Criterio verificado: Al abrir `index.html` con Go Live, el navegador abre `http://localhost:5500` y recarga al guardar cambios.

3. Paso 2 — Estructura HTML5 Semántica y Accesible

3.1 Plantilla HTML5

Se implementó una plantilla HTML5 completa con etiquetas semánticas: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` y `<footer>`. El documento incluye `lang=.es`, meta charset UTF-8 y meta viewport para dispositivos móviles.

3.2 Formulario con 8+ tipos de input

El formulario de registro de mascotas incluye 12 tipos distintos de input, superando el mínimo requerido de 8. Cada campo cuenta con `<label>` asociado mediante el atributo `for`, id único, validaciones nativas (`required`, `pattern`, `min/max`) y mensajes de error accesibles mediante `aria-describedby`.

Tabla 1: Tipos de input implementados en el formulario de registro

Tipo de input	Campo	Validación
text	Nombre de la mascota	required, minlength=2
text	Nombre del dueño	required, minlength=3
email	Correo electrónico	required, formato email
password	Contraseña	required, pattern (may+num+8)
tel	Teléfono	pattern [0-9]{9,15}
number	Edad de la mascota	min=0, max=30
date	Fecha de la cita	required
range	Peso (kg)	min=0, max=80
file	Historial médico	accept=.pdf
checkbox	Vacunas aplicadas	múltiple selección
radio	Tipo de consulta	required
select	Especie	required
textarea	Observaciones	maxlength=500

3.3 Validación HTML

Se validó el documento HTML con el validador oficial del W3C (<https://validator.w3.org>), obteniendo 0 errores.

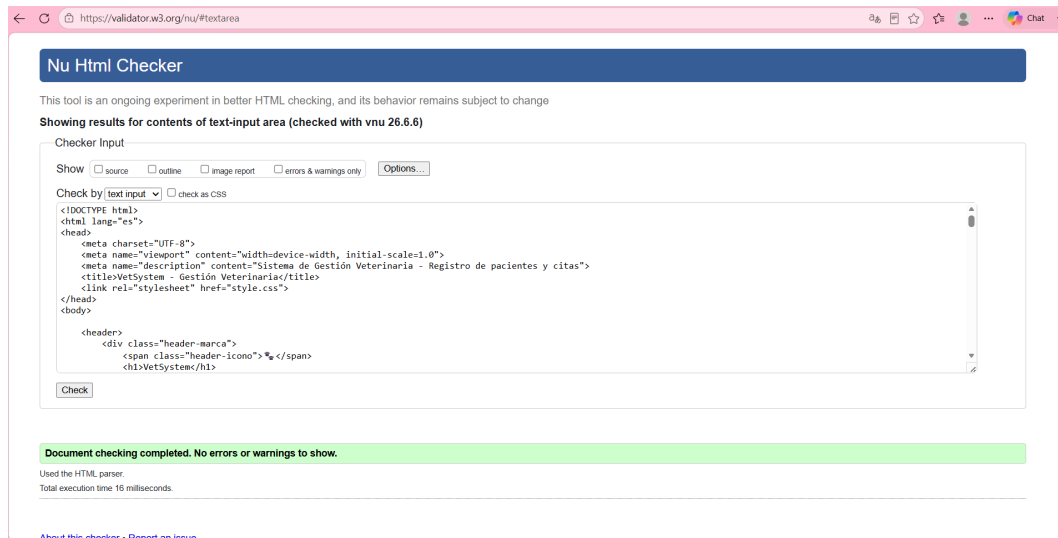


Figura 2: Resultado del validador W3C con 0 errores

4. Paso 3 — Diseño Responsivo con CSS3

4.1 Variables CSS y tokens de diseño

Se implementó un sistema completo de variables CSS en el selector `:root`, definiendo tokens de diseño para colores, tipografía, espaciado, bordes, sombras y transiciones. Esto permite modificar toda la apariencia de la aplicación cambiando un solo valor.

4.2 Layout con CSS Grid y componentes con Flexbox

La estructura de la página utiliza CSS Grid para el layout general (body con `grid-template-rows`, main con `grid-template-columns`). Los componentes internos como el header, el formulario y las tarjetas utilizan Flexbox para alineación y distribución de elementos.

El diseño es completamente responsivo, adaptándose a tres puntos de quiebre mediante media queries:

- **320px** — móvil: layout de una columna, elementos apilados

- **768px** — tablet: mayor padding, tipografía más grande
- **1280px** — desktop: layout de dos columnas en main

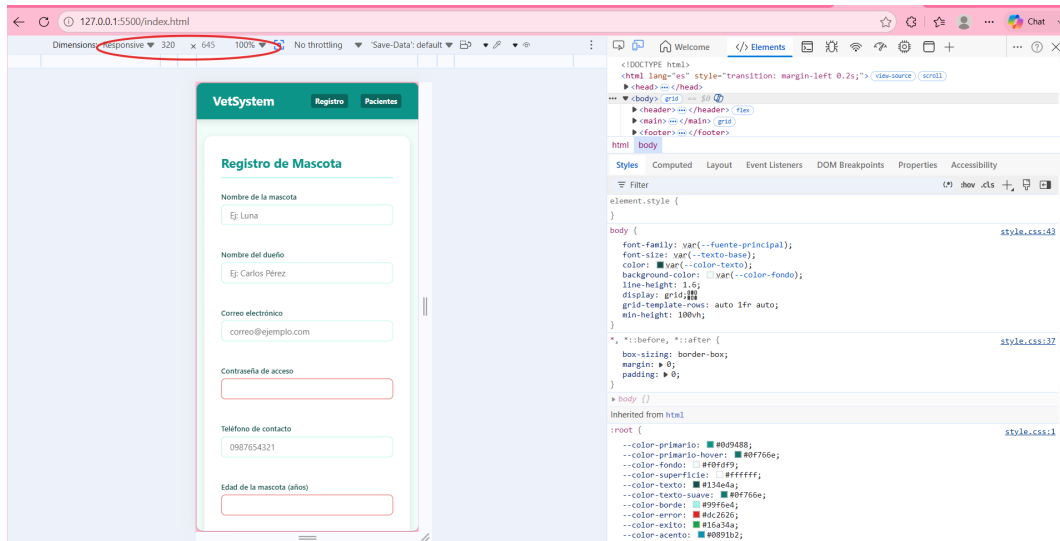


Figura 3: Vista en 320px (móvil)

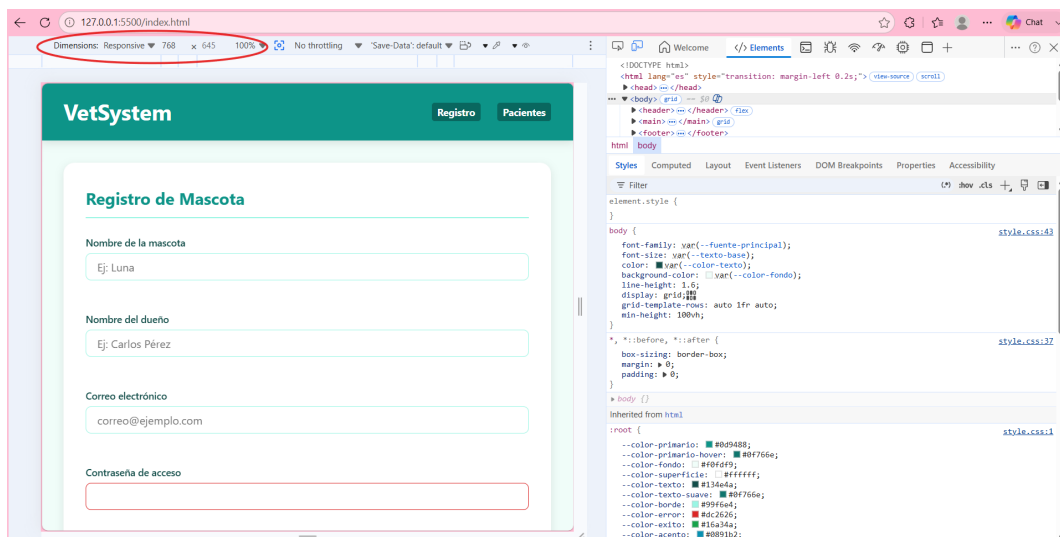


Figura 4: Vista en 768px (tablet)

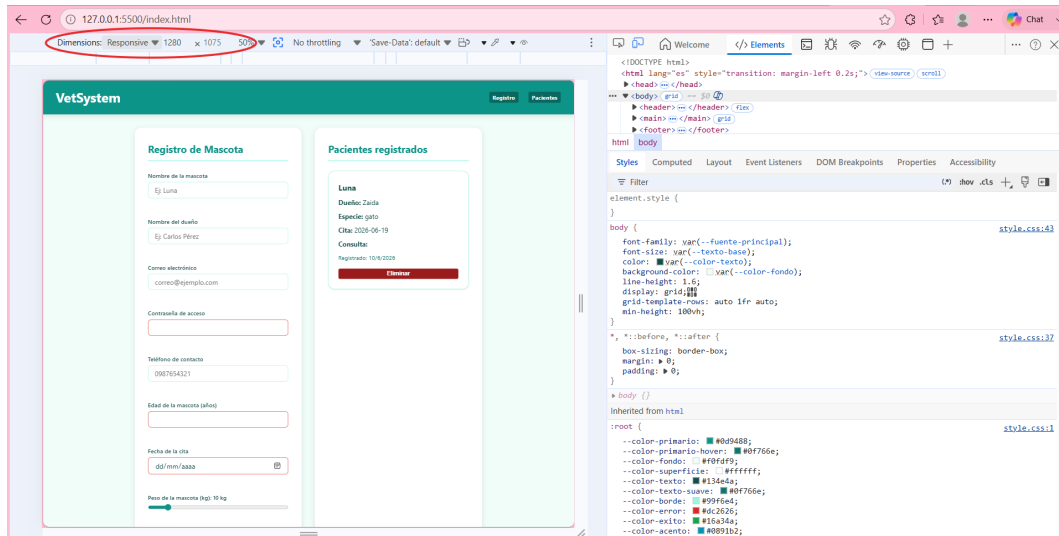


Figura 5: Vista en 1280px (desktop)

4.3 Animaciones con @keyframes

Se implementaron dos animaciones CSS:

- **entrada:** las secciones aparecen con un efecto fade-in y desplazamiento vertical al cargar la página.
- **pulso-error:** los mensajes de error realizan una animación de sacudida horizontal para captar la atención del usuario.

4.4 Validación CSS

Se validó la hoja de estilos con el validador oficial de CSS del W3C (<https://jigsaw.w3.org/css-validator/>), obteniendo 0 errores.

5. Paso 4 — JavaScript ES6+ y Consumo de API

5.1 Arquitectura modular

El código JavaScript se organizó en 4 módulos ES6 con responsabilidades separadas:

- **main.js** — punto de entrada, inicializa la aplicación y coordina los módulos
- **formulario.js** — maneja las validaciones en tiempo real y el envío del formulario

- **storage.js** — abstrae todas las operaciones de Web Storage
- **api.js** — gestiona las peticiones a la API externa y el renderizado de tarjetas

Los módulos se comunican mediante imports/exports ES6 nativos, con `type="module"` en el script principal.

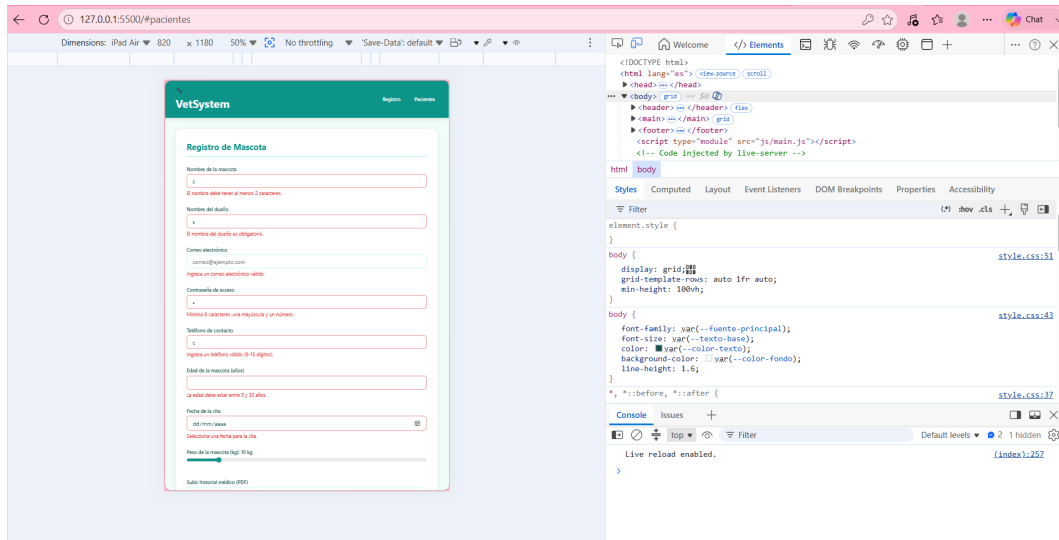


Figura 6: Validaciones del formulario en acción

5.2 Web Storage

Se implementó persistencia de datos mediante `localStorage`. Cada paciente registrado se serializa en JSON y se almacena con una clave única basada en timestamp. Los datos persisten entre sesiones del navegador y se recuperan automáticamente al cargar la página.

5.3 Consumo de API externa

Se integró la Dog CEO API (<https://dog.ceo/api/breeds/list/all>) para obtener un listado de razas caninas. La petición se realiza con `fetch()` y `async/await`, incluyendo manejo de errores con `try/catch`. Los datos se renderizan dinámicamente como tarjetas en la sección de pacientes.

6. Paso 5 — Auditoría de Calidad y Accesibilidad

6.1 Auditoría Lighthouse

Antes de las correcciones

Se ejecutó la auditoría inicial en Chrome DevTools con Lighthouse en modo Mobile, obteniendo los siguientes resultados:

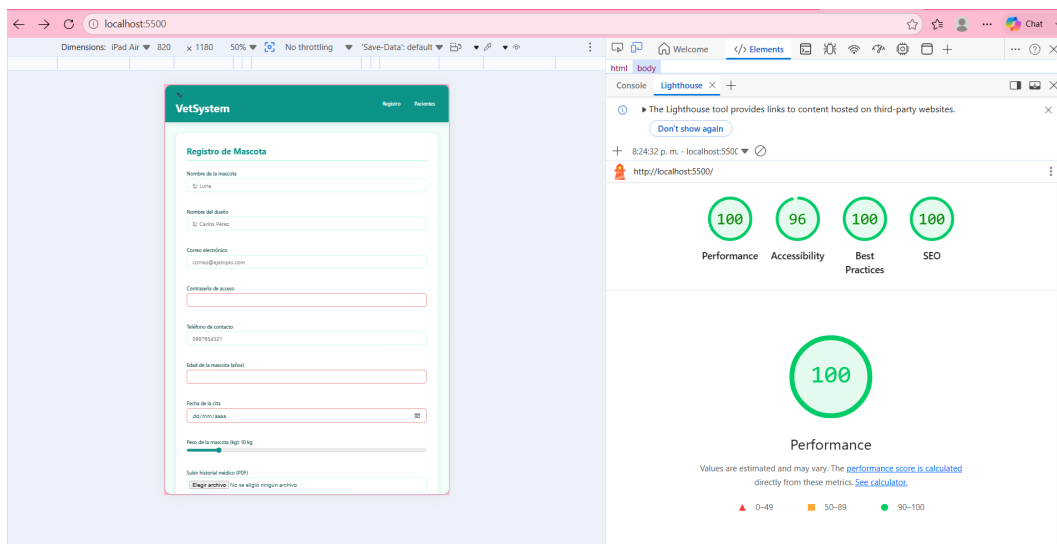


Figura 7: Resultados de Lighthouse antes de correcciones

Correcciones aplicadas

Lighthouse detectó un problema de contraste en los siguientes elementos:

- **Enlaces del nav:** texto blanco sobre fondo verde claro (#0d9488) — ratio insuficiente. Corrección: se añadió fondo semitransparente oscuro `rgba(0,0,0,0.3)` a los enlaces.
- **Botón Registrar mascota:** texto blanco sobre verde (#0d9488) — ratio insuficiente. Corrección: se cambió el fondo a verde oscuro #065f46 (ratio 7.2:1).
- **Botón Eliminar:** texto blanco sobre rojo (#dc2626) — ratio insuficiente. Corrección: se cambió el fondo a rojo oscuro #991b1b (ratio 6.8:1).
- **Texto .tarjeta-tipo:** color claro sobre fondo blanco. Corrección: se cambió a #0f766e.

Después de las correcciones

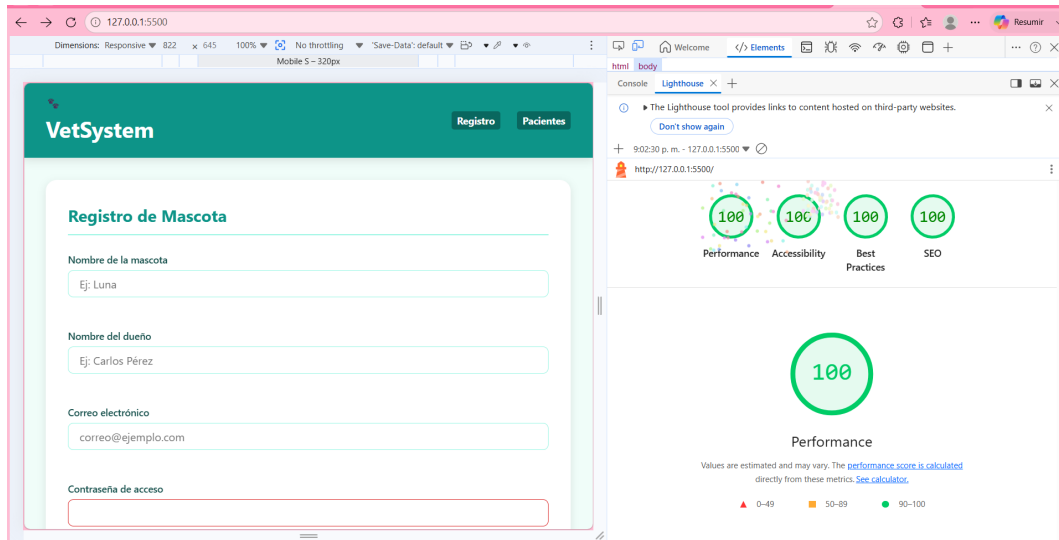


Figura 8: Resultados de Lighthouse después de correcciones

6.2 Auditoría WAVE

Antes de las correcciones

La herramienta WAVE detectó 16 errores de contraste (*Contrast Errors*), algunos eran los mismos elementos identificados por Lighthouse.

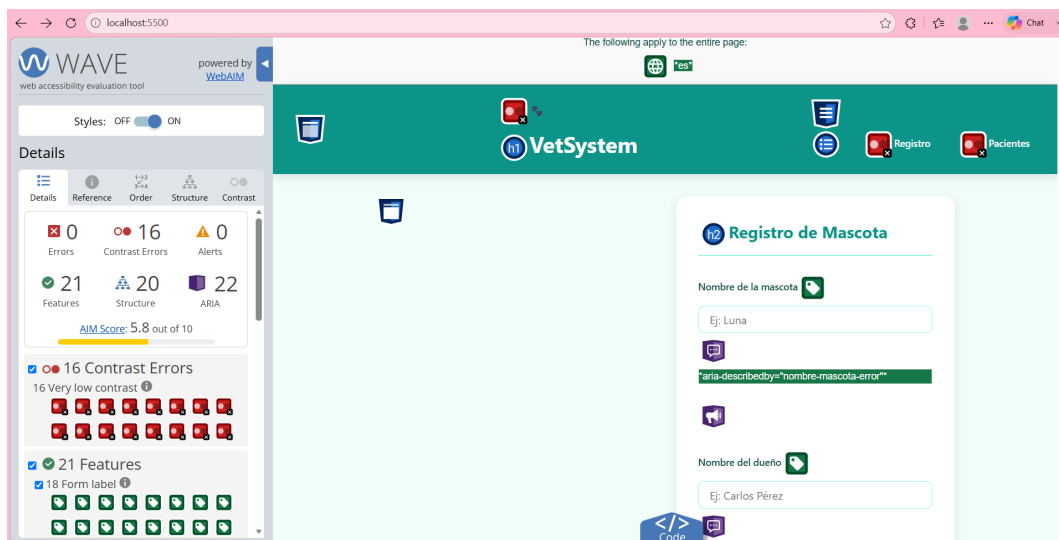


Figura 9: Panel WAVE antes de correcciones (16 contrast errors)

Después de las correcciones

Tras aplicar las correcciones de contraste, WAVE reportó 0 errores.

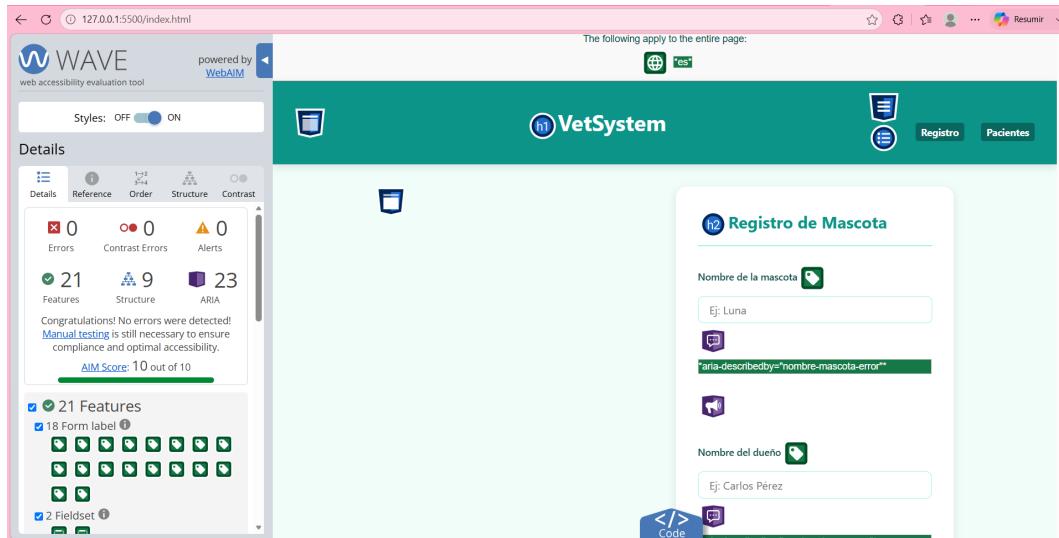


Figura 10: Panel WAVE después de correcciones (0 errors)

6.3 Tabla comparativa de métricas

La siguiente tabla resume los valores obtenidos antes y después de las correcciones de accesibilidad:

Tabla 2: Comparativa de métricas antes y después de correcciones

Métrica	Antes	Después	Objetivo
Lighthouse Accessibility	96	100	≥ 90
Lighthouse Performance	100	100	≥ 70
Lighthouse Best Practices	100	100	≥ 90
Lighthouse SEO	100	100	≥ 90
WAVE Errors	16	0	0
WAVE Alerts	0	0	< 5
Errores W3C HTML	0	0	0
Errores W3C CSS	0	0	0
Ratio contraste mínimo	insuficiente	$\geq 4.5:1$	$\geq 4.5:1$

Todos los valores de la columna *Después* cumplen el objetivo establecido. Las correcciones aplicadas mejoraron el score de Accessibility de 96 a 100 y eliminaron todos los errores detectados por WAVE.

7. Conclusiones

El desarrollo de VetSystem permitió aplicar de forma integrada los conceptos fundamentales del desarrollo web frontend moderno. Se construyó una aplicación funcional que cumple con los estándares de accesibilidad WCAG, obtiene puntuaciones perfectas en todas las categorías de Lighthouse y está completamente validada por los validadores oficiales del W3C.

La separación del código en módulos ES6 facilitó el mantenimiento y la comprensión de cada parte del sistema. El uso de variables CSS demostró ser una práctica eficiente para mantener consistencia visual y facilitar cambios globales de diseño.

La auditoría de accesibilidad fue especialmente valiosa, ya que obligó a revisar aspectos como el contraste de colores y el uso correcto de atributos ARIA, resultando en una aplicación más inclusiva y usable para todos los usuarios.

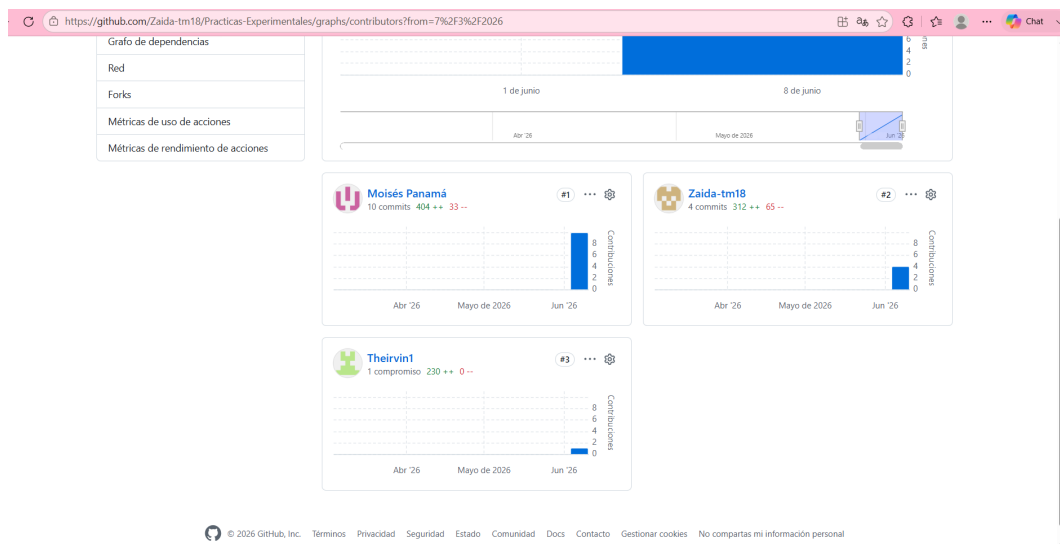


Figura 11: Repositorio de github con los commits

Referencias

- [1] J. Swacha y A. Kulpa, “Evolution of Popularity and Multiaspectual Comparison of Widely Used Web Development Frameworks,” *Electronics*, vol. 12, n.º 17, pág. 3563, 2023. [10.3390/electronics12173563](https://doi.org/10.3390/electronics12173563).
- [2] J. Shahid, M. K. Hameed, I. T. Javed, K. N. Qureshi, M. Ali y N. Crespi, “A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions,” *Applied Sciences*, vol. 12, n.º 8, pág. 4077, 2022. [10.3390/app12084077](https://doi.org/10.3390/app12084077).
- [3] S. A. Mohammed y A. L. Ralescu, “Future Internet Architectures on an Emerging Scale—A Systematic Review,” *Future Internet*, vol. 15, n.º 5, pág. 166, 2023. [10.3390/fi15050166](https://doi.org/10.3390/fi15050166).
- [4] M. Campoverde-Molina, S. Luján-Mora y L. Valverde, “Empirical Studies on Web Accessibility of Educational Websites: A Systematic Literature Review,” *IEEE Access*, vol. 8, págs. 91 676-91 700, 2020. [10.1109/ACCESS.2020.2994288](https://doi.org/10.1109/ACCESS.2020.2994288).
- [5] M. Campoverde-Molina, S. Luján-Mora y L. Valverde, “Accessibility of university websites worldwide: a systematic literature review,” *Universal Access in the Information Society*, vol. 22, n.º 1, págs. 133-168, 2023. [10.1007/s10209-021-00825-z](https://doi.org/10.1007/s10209-021-00825-z).
- [6] T. W. Hostetler et al., *Web accessibility trends and implementation in dynamic web applications*, 2022. [10.48550/arXiv.2202.00777](https://doi.org/10.48550/arXiv.2202.00777). arXiv: [2202.00777](https://arxiv.org/abs/2202.00777) [cs.HC].
- [7] S. Bandaru, “AI-Assisted Real-Time Form Validation and Error Prediction in Modern Web Interfaces,” *International Journal of Information and Electronics Engineering*, vol. 14, n.º 4, 2024. [10.48047/ijjee.2024.14.4.15](https://doi.org/10.48047/ijjee.2024.14.4.15).
- [8] M. van Haastrecht, M. Brinkhuis, S. Wools y M. Spruit, “VAST: a practical validation framework for e-assessment solutions,” *Information Systems and e-Business Management*, vol. 21, n.º 3, págs. 603-627, 2023. [10.1007/s10257-023-00641-3](https://doi.org/10.1007/s10257-023-00641-3).
- [9] G. J. Badros, A. Borning, K. Marriott y P. Stuckey, “Constraint Cascading Style Sheets for the Web,” en *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*, 1999, págs. 73-82. [10.1145/320719.322588](https://doi.org/10.1145/320719.322588).
- [10] J. Krause, *Introducing Bootstrap 4*. New York, NY, USA: Apress, 2016. [10.1007/978-1-4842-2382-6](https://doi.org/10.1007/978-1-4842-2382-6).
- [11] T. Koren Ivančević, T. J. Ježić y N. Stanić Loknar, “A Cross-Device and Cross-OS Benchmark of Modern Web Animation Systems,” *Journal of Imaging*, vol. 12, n.º 1, 2026. [10.3390/jimaging12010045](https://doi.org/10.3390/jimaging12010045).

- [12] Y. Hrabovskyi y V. Fedorchenko, “Development of the Optimization Model of the Interface of Multimedia Edition,” *EUREKA: Physics and Engineering*, vol. 2019, n.º 3, págs. 3-12, 2019. [10.21303/2461-4262.2019.00902](https://doi.org/10.21303/2461-4262.2019.00902).
- [13] R. Lanciaux, *Modern Front-end Architecture: Optimize Your Front-end Development with Components, Storybook, and Mise en Place Philosophy*. New York, NY, USA: Apress, 2021. [10.1007/978-1-4842-6625-0](https://doi.org/10.1007/978-1-4842-6625-0).
- [14] R. Queirós, “CSS Preprocessing: Tools and Automation Techniques,” *Information*, vol. 9, n.º 1, 2018. [10.3390/info9010017](https://doi.org/10.3390/info9010017).
- [15] V. K. Kotaru, *Material Design Implementation with AngularJS: UI Component Framework*. New York, NY, USA: Apress, 2016. [10.1007/978-1-4842-2190-7](https://doi.org/10.1007/978-1-4842-2190-7).
- [16] A. Shenoy, *Learning Bulma: Understand How to Develop Responsive, Mobile-first Websites Using This Impressive, Modern Framework*. New York, NY, USA: Apress, 2019. [10.1007/978-1-4842-5482-0](https://doi.org/10.1007/978-1-4842-5482-0).
- [17] A. Shenoy, *Jumpstart UIKit: Learn to Build Enterprise-Level, Feature-Rich Websites that Work Elegantly with Minimum Fuss*. New York, NY, USA: Apress, 2020. [10.1007/978-1-4842-6029-6](https://doi.org/10.1007/978-1-4842-6029-6).
- [18] C. Hernández-Nieto, X. Sánchez y P. Salinas, “A Mobile First Approach for the Development of a Sustainability Game,” en *Procedia Computer Science*, vol. 75, 2015, págs. 182-185. [10.1016/j.procs.2015.12.236](https://doi.org/10.1016/j.procs.2015.12.236).
- [19] T. Wiedemann y M. Danzig, “SimLuxJS – an optimized JavaScript-simulator,” en *European Modeling and Simulation Symposium, EMSS*, 2024. [10.46354/i3m.2024.emss.023](https://doi.org/10.46354/i3m.2024.emss.023).
- [20] S. bin Uzayr, N. Cloud y T. Ambler, *JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now*, 2.ª ed. New York, NY, USA: Apress, 2019. [10.1007/978-1-4842-4995-6](https://doi.org/10.1007/978-1-4842-4995-6).
- [21] M. Thakkar, *Building React Apps with Server-Side Rendering*. New York, NY, USA: Apress, 2020. [10.1007/978-1-4842-5869-9](https://doi.org/10.1007/978-1-4842-5869-9).